Estimation and Optimization Layers in Binary Classification with Naive Bayes, Logistic Regression, and Support Vector Machine

Jihoon Choi [*1] **Dae Young Kim** [*1]

Abstract

This project explores how statistical estimation principles connect with numerical optimization techniques in binary classification. Modern machine learning libraries often hide the underlying optimization steps, treating them as black boxes. To better understand how learning actually occurs, we implemented three classifiers from scratch using the UCI Default of Credit Card Clients dataset ($N = 30{,}000$): Gaussian Naive Bayes (GNB) as a closed-form statistical baseline, and Logistic Regression and Support Vector Machine (SVM) as optimization-driven models. We focused in particular on how different solvers (from fixed-step Gradient Descent to second-order methods such as BFGS) shape convergence behavior, stability, and computational efficiency.

Our experiments show a clear trade-off between computational cost and the number of iterations required for convergence. In Logistic Regression with L2 regularization, BFGS converged in just 10 iterations, while fixed-step Gradient Descent needed 93 iterations to reach the same loss level (0.6099). Similarly, for SVM, smoothing the objective (Squared Hinge loss) allowed BFGS to converge in only 18 iterations, drastically outperforming the subgradient method on the standard Hinge loss. Ultimately, SVM achieved the highest test accuracy of 81.2%, slightly edging out Logistic Regression (80.6%) and significantly outperforming the generative GNB baseline (77.1%), underscoring the benefits of discriminative models when paired with effective optimization strategies.

1. Introduction

Binary classification is a fundamental task in machine learning, typically approached through two distinct paradigms: statistical estimation and numerical optimization. While modern libraries like `scikit-learn` abstract away the complexity of these methods, treating them as black boxes often obscures the internal mechanics of learning. In this project, we want to open that black box by implementing core algorithms from first principles.

We focus on predicting credit card defaults using the UCI dataset. To contrast the two learning paradigms, we implement Gaussian Naive Bayes (GNB) as a closed-form statistical baseline and compare it against Logistic Regression and Support Vector Machines (SVM), both of which require iterative numerical optimization. Our primary contribution is a rigorous comparative analysis of optimization solvers (ranging from first-order Gradient Descent to second-order Quasi-Newton methods) evaluating them not just on accuracy, but on convergence speed, stability, and computational cost.

Summary of Results: Our experiments reveal a meaningful trade-off between algorithmic complexity and convergence efficiency. We found that utilizing second-order curvature information (BFGS) reduced the training iterations for Logistic Regression from 93 (Fixed-Step GD) to just 10. Similarly, for SVM, smoothing the non-differentiable Hinge Loss allowed for the use of fast gradient-based solvers, achieving the highest test accuracy of 81.2%. These findings underscore that the choice of optimizer is not merely a computational detail but a fundamental modeling decision, determining the structural sparsity and robustness of the final decision boundary.

2. Problem Statement

The goal of this study is to predict whether a credit card client will default on their payment in the next month. Formally, this is a binary classification task in which the input X contains client features (demographic and financial information), and the output y indicates default status ($y \in \{0, 1\}$ or $y \in \{-1, +1\}$).

Although predictive accuracy is an important outcome, the central focus of this work is the learning process itself. We use this task as a case study to contrast two fundamentally different approaches to parameter estimation:

- **Generative Approach (Estimation):** Estimating parameters under assumed probability distributions (e.g., Gaussian) without relying on iterative error minimization. This serves as our closed-form baseline.
- **Discriminative Approach (Optimization):** Defining a convex objective function and using iterative numerical methods to locate its global minimum.

Our most important goal is to understand how algorithmic choices, particularly the selection of an optimizer and the formulation of the loss function, affect the stability and speed of learning. Specifically, we investigate two key trade-offs in optimization:

1. **Order of Information:** Whether the extra computational effort required for Hessian approximation in second-order methods (BFGS) is justified by the reduction in iteration count compared to first-order Gradient Descent.
2. **Loss Function Smoothness:** How the mathematical formulation of the objective (e.g., non-differentiable Hinge Loss vs. smooth Squared Hinge Loss in SVM) determines the choice of viable solvers and impacts convergence behavior.

3. Data

3.1. Dataset Description

For this project, we use the UCI Default of Credit Card Clients dataset (Yeh & hui Lien, 2009), which is a standard benchmark in the field of financial risk analysis. The dataset contains 30,000 observations collected. The target variable is `def_pay`, indicating whether the client defaulted (1) or paid (0) in the next month.

The feature set consists of 23 variables, categorized into three groups:

- **Demographics:** `limit_bal` (credit limit), `sex`, `education`, `marriage`, and `age`.
- **Repayment History:** Variables `pay_0` through `pay_6` represent the repayment status from September to April. A positive value indicates a delay in months (e.g., 2 means a two-month delay), while zero or negative values indicate on-time payment.
- **Financial Amounts:** `bill_amt1` – `bill_amt6` represent the monthly bill statement amounts, and `pay_amt1` – `pay_amt6` represent the amounts paid in the previous months.

We conducted an exploratory analysis to understand the data distribution and identify potential challenges for modeling. First, the dataset is imbalanced; only about 22% of the samples are positive (default) cases. This imbalance suggests that accuracy alone might be misleading, so we also monitored metrics like F1-score and AUC.

Through statistical tests, we found significant relationships between features and the target. Using Chi-square tests for categorical variables, we observed that repayment status is the strongest predictor. Clients who delayed payment by 2 months or more had a drastically higher probability of default, with standardized residuals up to 51.6 for `pay_0` = 2. Demographic features also showed significant trends: males had a 4.8 times higher default risk (residual 4.75), and those with high school education or below had a 4.5 times higher risk (residual 4.53).

For numerical features, t-tests and distributions showed significant differences: defaulters generally had lower credit limits ($p = 1.23 \times 10^{-189}$) and lower recent payments ($p < 1 \times 10^{-90}$), along with slightly lower recent bill amounts ($p < 0.05$).

Crucially, we observed high multicollinearity among the `bill_amt` and `pay` variables, with correlation coefficients ranging from 0.66 to 0.95. Such high correlations can lead to an ill-conditioned optimization problem, producing a loss landscape shaped like a narrow valley. This observation highlights the importance of using regularization in our Logistic Regression and SVM models to ensure numerical stability.

3.2. Preprocessing

Proper data preprocessing is essential for ensuring the numerical stability and efficient optimization. We applied the following pipeline to prepare the data for training:

To improve readability, we first converted variable names to human-readable forms. Categorical variables include gender (`sex`), education level (`education`), and marital status (`marriage`), along with the binary outcome variable (`def_pay`). Temporal predictors consist of the delinquency status variables `pay_1` through `pay_6`, which capture repayment status from the most recent to the sixth month prior, as well as billing (`bill_amt`) and payment amounts (`pay_amt`).

1. Categorical Encoding & Cleaning
Several categorical variables included undocumented labels. For example, the `education` feature included values 0, 5, and 6, which were not defined in the dataset documentation. We merged these rare categories into a single "Others" class

(assigned label 4). Undocumented values in `marriage` were handled in the same way. For the repayment status variables (`pay`), all non-delay codes (originally -2, -1, and 0) were consolidated into a single value (0). This change makes a clearer ordinal scale of delay severity, which is more suitable for linear models.

2. Train–Test Split
We divided the dataset into a training set (24,000 samples, 80%) and a test set (6,000 samples, 20%) using a fixed random seed. This ensures that all models (GNB, LR, SVM) are evaluated on the exact same unseen data for a fair comparison.

3. Standardization (Feature Scaling)
This step is particularly important for our optimization experiments. The raw features vary widely in scale; for instance, `age` ranges from roughly 20 to 80, while `limit_bal` can reach 1,000,000. Without scaling, optimization algorithms such as Gradient Descent face elongated loss contours (ill-conditioned landscape), which makes the algorithm to take slow, unstable, and zigzagging updates.

To address this, we applied standardization to all numerical features:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

This places features on a comparable scale (mean 0, variance 1), making a better-conditioned optimization landscape. This allows optimization algorithms to move more directly toward the minimum, improving both convergence speed and numerical stability.

4. Statistical Estimation Model: Gaussian Naive Bayes

4.1. Methodology and Formulation

To establish a performance baseline, we implemented Gaussian Naive Bayes (GNB). Unlike the optimization-based models in this project (LR, SVM), GNB relies on statistical estimation. Instead of iteratively minimizing an error function, it estimates the parameters of the data distribution directly in a single pass.

The core of the model is Bayes' Theorem combined with the Naive assumption of conditional independence among features. For a given class c and a feature vector $x = [x_1, \ldots, x_n]$, the posterior probability is proportional to the product of the class prior and the feature likelihoods.

Assuming each feature x_i follows a Gaussian distribution, the decision rule for predicting the class label $\hat{y}$ is formulated as:

$$\hat{y} = \arg\max_{c \in \{0,1\}} P(y = c) \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma_{ci}^2}} \exp\left(-\frac{(x_i - \mu_{ci})^2}{2\sigma_{ci}^2}\right)$$

To implement this, we calculated two sets of statistics directly from the training data:

- **Class Prior $P(y = c)$:** The relative frequency of each class (Default vs. No Default).
- **Gaussian Parameters (μ_{ci}, σ_{ci}^2):** The mean and variance of feature i given class c.

During prediction, we compute the log-posterior to avoid numerical underflow (since multiplying many small probabilities can result in zero). The class with the highest posterior probability is selected as the prediction.

4.2. Results

We evaluated our GNB implementation on the test set and compared it against `scikit-learn`'s `GaussianNB` to ensure validity.

Table 1. Gaussian Naive Bayes Performance (Test Set)

Metric	Our Implementation	Scikit-Learn
Accuracy	0.7713	0.7713
F1 Score	0.5206	0.5206
ROC-AUC	0.7471	0.7471
Log Loss	1.6005	1.6016

As shown in Table 1, our implementation matched the library performance exactly, confirming our code is correct. The accuracy was 77.13%. While this is a decent starting point, the F1-score is relatively low (0.52). This indicates that the Naive assumption that features are independent is likely violated. In our EDA, we saw strong correlations between bill amount variables. GNB cannot learn these interactions, which limits its performance compared to optimization-based models.

5. Optimization Model: Logistic Regression

5.1. Methodology & Formulation

After establishing the baseline, we moved to Optimization-based learning. Logistic Regression learns a decision boundary by minimizing a convex error function. Unlike GNB, it does not assume features are independent or normally distributed.

5.1.1. Objective Function

We formulated the problem as minimizing the Negative Log-Likelihood with L2 Regularization (Ridge). Regularization is particularly crucial here because of the multicollinearity we observed in the EDA; without it, the weights for highly correlated features (e.g., `bill_amt`) could become unstable.

The objective function $J(w)$ is defined as:

$$J(w) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\Big] + \frac{\lambda}{2}\|w\|^2$$

where N is the number of samples, w is the weight vector, and $\hat{y}_i = \sigma(w^T x_i)$ is the sigmoid probability. We set the regularization strength $\lambda = 1.0$ for our experiments.

To optimize this objective, we derived the gradient vector $\nabla J(w)$ analytically. This gradient is essential as it provides the direction of steepest ascent/descent for our solvers:

$$\nabla J(w) = \frac{1}{N} X^T(\hat{y} - y) + \lambda w$$

We implemented these computations using vectorized NumPy operations to ensure high performance during the iterative updates.

5.1.2. Optimization Algorithms

A key goal of this project was to open the black box of training. Instead of relying on a default solver, we implemented and compared four distinct optimization strategies, ranging from first-order to quasi-Newton methods.

- **Fixed-Step Gradient Descent (GD):** This is the simplest first-order method. It updates weights by moving in the opposite direction of the gradient ($w_{t+1} = w_t - \eta \nabla J$). While easy to implement, it is highly sensitive to the learning rate η. If η is too small, convergence is slow; if too large, it diverges.
- **Gradient Descent with Armijo Backtracking:** To address the limitations of fixed steps, we implemented an adaptive line search. At each iteration, the algorithm checks the Armijo condition to ensure the chosen step size yields a sufficient decrease in loss. If the step is too aggressive, it backtracks (shrinks the step size). This prevents the algorithm from overshooting and oscillating around the minimum.
- **Nonlinear Conjugate Gradient (NLCG):** Standard GD often zigzags when the loss landscape creates a narrow valley. NLCG improves upon this by enforcing that the new search direction is conjugate to previous ones, preserving past progress. We utilized the Polak-Ribière update rule, which is generally considered superior for non-quadratic objective functions.
- **BFGS (Quasi-Newton Method):** This is the most advanced solver we implemented. Unlike GD which only sees the slope (first derivative), BFGS approximates the Hessian matrix (second derivative) to understand the curvature of the loss landscape. This allows the algorithm to determine not just the direction, but the optimal scale of the step, effectively accounting for the geometry of the function.

5.2. Results & Analysis

We trained the model using all four algorithms on the training set. The stopping criterion was strict: the process continued until the gradient norm fell below 10^{-6}.

5.2.1. Convergence Comparison

Table 2 summarizes the performance of each algorithm. The difference in efficiency was substantial.

Table 2. Convergence Comparison of Optimization Algorithms

Optimizer	Iters	$\|\nabla \ell\|$	Final Loss	Test Acc.
Fixed GD	93	9.40×10^{-7}	0.6099	0.806
GD + Armijo	13	9.51×10^{-7}	0.6099	0.806
NLCG (PR+)	18	4.35×10^{-7}	0.6099	0.806
BFGS	**10**	$\mathbf{1.45 \times 10^{-7}}$	**0.6099**	**0.806**

We observed a significant disparity in convergence speed across the solvers. Fixed-Step Gradient Descent required 93 iterations to reach the tolerance threshold, but BFGS converged in just 10 iterations, achieving the lowest final gradient norm ($\|\nabla \ell\| \approx 1.45 \times 10^{-7}$). Importantly, despite these differences in speed, all four algorithms converged to the identical final loss (0.6099) and test accuracy (0.806). This confirms that while the *path* to the minimum varies greatly in efficiency, all solvers successfully located the same global optimum.

5.2.2. Analysis of Convergence Behavior

The dramatic difference in iteration counts, from 93 (Fixed GD) to 10 (BFGS), can be attributed to the geometry of the loss landscape and how each algorithm interacts with it.

- **The Impact of Ill-Conditioning:** As observed in the EDA section, our dataset has highly correlated features (e.g., `bill_amt` variables). High correlation creates an ill-conditioned Hessian matrix, transforming the loss landscape into a narrow, elongated valley (a canyon).

- **Limitations of First-Order Methods (Fixed GD):** Standard Gradient Descent uses only the gradient, which is perpendicular to the contour lines. In a narrow valley, the steepest descent direction points mostly across the valley rather than down towards the minimum. This causes Fixed GD to oscillate back and forth (zigzagging) across the canyon walls, resulting in very slow convergence (93 iterations).
- **Efficiency of Adaptive Steps (Armijo):** Even without curvature information, Armijo Line Search provided a 7x speedup (93 $\rightarrow$ 13 iterations). By dynamically reducing the step size when the error doesn't decrease sufficiently, it prevents the overshooting that causes severe zigzagging. It allows the algorithm to take larger steps in flat regions and smaller, safer steps in steep regions.
- **Superiority of Second-Order Methods (BFGS):**

  BFGS achieved the fastest convergence (10 iterations) because it approximates the inverse Hessian matrix (H^{-1}). This matrix captures the curvature of the loss function, effectively rescaling the geometry of the valley to look more spherical. Instead of blindly following the steepest slope, BFGS calculates a Newton direction that points directly toward the bottom of the valley, enabling it to reach the optimum in very few steps despite the high feature correlations.

5.2.3. Validation and Discrepancy Analysis

All four of our implementations converged to the exact same loss (approximately 0.6099) and test accuracy (0.8057). This consistency across different solvers (from first-order gradient descent to quasi-Newton methods) confirmed that our optimization logic and gradient derivations were correct.

However, when we compared our results with `scikit-learn`'s `LogisticRegression`, we noticed a slight difference in accuracy (0.8057 vs. 0.8168). To understand this discrepancy, we investigated the underlying formulation differences. We found two key factors:

- **Objective Function Scaling (Major Factor):** Our implementation minimizes the average loss over the dataset ($\frac{1}{N}\sum L_i$), whereas `scikit-learn` minimizes the sum of losses weighted by a regularization parameter C ($C\sum L_i$). Mathematically, our regularization parameter $\lambda = 1.0$ corresponds to an extremely small C value in `scikit-learn` ($C \approx \frac{1}{N\lambda} \approx 4 \times 10^{-5}$). This means our model was subject to much stronger regularization, leading to slight underfitting compared to the library default.
- **Intercept Regularization (Minor Factor):** In our code, we applied the L2 penalty to the entire weight vector w, including the bias (intercept) term w_0. In contrast, `scikit-learn` and many statistical packages do not penalize the intercept by default. Penalizing the bias tends to shrink it toward zero, pushing the model's default prediction probability closer to 0.5. Given that our dataset is imbalanced (22% positive), an unregularized bias allows the model to better adjust its baseline probability to match the class prior, slightly improving accuracy.

After adjusting our implementation formulation to match `scikit-learn`'s (removing the penalty on the bias and aligning the regularization strength), our implementation achieved the same accuracy. This analysis was crucial because it proved that the performance gap was not an optimization failure, but a result of distinct modeling choices.

6. Optimization Model: Support Vector Machine

6.1. Methodology & Formulations

6.1.1. Primal Formulation and Loss Functions

To complement the Logistic Regression baseline, this study implemented a Support Vector Machine (SVM) optimized directly in the *Primal formulation*. Unlike traditional implementations that solve the Dual problem to leverage kernel tricks, the Primal approach optimizes the weight vector w and bias b directly in the feature space. This formulation is particularly advantageous for large-scale datasets ($N \gg d$) where the computational cost scales with the number of features rather than the number of samples. Following the framework (Hastie et al., 2009), the objective was defined as the minimization of the regularized empirical risk:

$$\min_{w,b} J(w,b) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N} \ell(y_i, f(x_i))$$

This objective function comprises two competing terms that balance model complexity against training accuracy:

- **Regularization Term ($\frac{1}{2}\|w\|^2$):** Minimizing the squared L2 norm of the weight vector is geometrically equivalent to maximizing the separation margin ($2/\|w\|$) between classes. This term acts as a stabilizer, preventing overfitting by penalizing large weights and enforcing a flatter decision boundary.
- **Hyperparameter C (Soft-Margin Control):** The parameter C governs the trade-off between maximizing the margin and minimizing classification errors. In this

implementation, C was set to 10. This finite, moderate penalty establishes a Soft-Margin SVM, which explicitly permits some training data points to violate the margin constraints (or be misclassified) in exchange for a wider, more robust decision boundary. Unlike a Hard-Margin formulation (where effectively $C \to \infty$), this soft-margin approach ensures the optimization remains feasible even when the data is not perfectly linearly separable due to noise.

- **Empirical Loss Term ($\sum \ell(\dots)$):** This term sums the penalties for all misclassified samples or samples falling within the margin. The specific shape of the penalty is determined by the loss function specification $\ell(z)$ (either Standard Hinge or Squared Hinge), as detailed below.

To investigate the relationship between objective function smoothness and optimization stability, two distinct specifications of the loss function $\ell(z)$ were analyzed. For notation, we define z_i as the *functional margin* of the x_i:

$$z_i = y_i f(x_i) = y_i(w^\top x_i + b)$$

where $z_i > 0$ indicates a correct classification (matching sign) and $z_i < 1$ indicates a margin violation.

The first specification, Squared Hinge Loss ($\ell(z_i) = \max(0, 1 - z_i)^2$), renders the objective function continuously differentiable (C^1 smooth) everywhere, including the margin boundary. This smoothness permits the use of second-order optimization methods that exploit curvature information.

The second specification, Standard Hinge Loss ($\ell(z_i) = \max(0, 1 - z_i)$), retains the non-differentiable discontinuity at the margin boundary ($z_i = 1$). While this non-smoothness precludes strict gradient-based methods, theoretical literature suggests it induces sparsity in the model parameters, effectively ignoring correctly classified sample points where $z_i \geq 1$.

6.1.2. Optimization Strategies

To investigate the impact of solver sophistication on model performance, we implemented three distinct optimization algorithms from first principles, adhering to the theoretical frameworks (Nocedal & Wright, 2006; Kochenderfer & Wheeler, 2019). The selection of these solvers was strictly based on the differentiability of the loss formulations defined above. For the smooth Squared Hinge objective, we employed both Fixed-Step Gradient Descent (as a first-order baseline) and the BFGS quasi-Newton method to evaluate the benefits of exploiting second-order curvature information. Conversely, for the non-smooth Standard Hinge objective, we utilized Stochastic Gradient Descent (SGD), relying on sub-gradients to navigate the discontinuity at the margin boundary where the standard gradient is undefined.

- **Fixed-Step Gradient Descent (Baseline)** As a baseline for numerical precision, we implemented a standard first-order Gradient Descent (GD) algorithm on the smooth Squared Hinge objective. At each iteration k, the weight vector w is updated in the direction of the negative gradient:

  $$w_{k+1} = w_k - \eta \nabla J(w_k)$$

  where η is a fixed step size (learning rate). This method utilizes the full dataset to compute the exact gradient ∇J at every step. While theoretically guaranteed to converge for convex, smooth functions with an appropriate η, fixed-step GD is known to suffer from poor convergence rates on ill-conditioned problems, often exhibiting oscillatory zig-zagging behavior in steep valleys. We utilized a strict termination tolerance of $\|\nabla J\|_2 < 10^{-6}$ to ensure the solver reached the true mathematical minimum.

- **BFGS (Quasi-Newton Method)** To address the ill-conditioning and linear convergence speed of standard GD, we implemented the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm. As a quasi-Newton method, BFGS seeks to approximate the Newton step $w_{k+1} = w_k - H^{-1}\nabla J(w_k)$ without the computational expense of calculating and inverting the true Hessian matrix H.

  Our implementation iteratively updates an approximate inverse Hessian matrix $B_k \approx (\nabla^2 J)^{-1}$ using gradient information from previous steps. To ensure global convergence and sufficient objective decrease, we coupled the BFGS direction with an *Armijo Backtracking Line Search*. At each iteration, the step size α is dynamically contracted until the sufficient decrease condition is met:

  $$J(w_k + \alpha p_k) \leq J(w_k) + c_1 \alpha \nabla J_k^T p_k$$

  This combination allows the solver to capture curvature information and achieve superlinear convergence on the smooth Squared Hinge surface.

- **Stochastic Gradient Descent (SGD) with Annealing** For the non-smooth Standard Hinge loss, gradients are undefined at the margin boundary ($1 - y_i f(x_i) = 0$). Consequently, we implemented a sub-gradient descent method. To improve computational efficiency on large-scale data, we utilized a Stochastic formulation (batch size $= 1$), updating weights based on a single random sample at a time.

  Because sub-gradient methods with constant step sizes are not guaranteed to converge to the optimum (often

oscillating within a noise floor), we implemented a *Learning Rate Annealing* schedule based on inverse time decay. The step size at iteration t is defined as:

$$\eta_t = \frac{\eta_0}{1 + k \cdot t}$$

where η_0 is the initial learning rate and k is a decay hyperparameter. This schedule allows the algorithm to traverse the loss landscape quickly in early epochs while dampening variance as it approaches the non-differentiable minimum.

6.2. Results & Analysis

6.2.1. Results and Discussion

Table 3 summarizes the convergence metrics and predictive performance of each strategy.

Table 3. Convergence and Performance of Primal SVM Solvers

Optimizer	Iterations	Final $\|\nabla J\|$	SVs	Accuracy
Fixed-Step GD	3,107	9.97×10^{-7}	23,602	0.812
BFGS	**18**	1.89×10^{-1}	23,597	0.812
Stochastic GD	99 (Epochs)	6.67×10^{-2}	**9,769**	0.807

The experimental results highlight distinct trade-offs between computational efficiency, numerical precision, and model complexity. Key observations and interpretations include:

- **Convergence Efficiency (BFGS vs. GD):** The BFGS algorithm demonstrated superior optimization efficiency, converging in only 18 iterations compared to over 3,000 for Fixed-Step GD. This superlinear convergence confirms the benefit of exploiting second-order curvature information (via the inverse Hessian approximation) on the smooth Squared Hinge surface. In contrast, the first-order GD method exhibits a linear convergence rate, struggling to navigate the ill-conditioned valley of the objective function without curvature data.
- **Numerical Stability and Precision:** While Fixed-Step GD achieved high numerical precision ($\|\nabla J\| \approx 10^{-7}$), BFGS terminated with a relatively higher gradient norm ($\approx 10^{-1}$). This early termination is likely due to the dense nature of the solution (98% active support vectors). With nearly all data points contributing to the gradient, the objective surface becomes numerically noisy, causing the line search to fail in satisfying the Wolfe conditions at extremely high precision. However, this did not negatively impact predictive accuracy.
- **Sparsity vs. Density (The Hinge Loss Effect):** A fundamental disparity was observed in model complexity. The smooth Squared Hinge solvers (GD/BFGS) retained $\approx 98\%$ of the training data as Support Vectors, whereas the non-smooth Standard Hinge solver (SGD) retained only $\approx 41\%$. The Squared Hinge loss ($\max(0, 1 - z)^2$) imposes a quadratic penalty on all margin violations, forcing the model to adjust the decision boundary for every single noisy data point, resulting in a dense solution. Conversely, the Standard Hinge loss ($\max(0, 1 - z)$) yields a zero gradient for all correctly classified points outside the margin. This effectively filters out safe samples, inducing sparsity and acting as a robust regularizer against overfitting.
- **Generalization Performance:** Despite the drastic difference in model complexity (9,769 SVs vs. 23,602 SVs), the Stochastic GD model achieved a test accuracy of 0.807, which is statistically equivalent to the 0.812 of the dense models. The marginal gain in accuracy ($< 0.5\%$) from the dense solvers does not justify their computational cost and complexity. For noisy financial data, the sparse solution provided by SGD is preferable, as it achieves comparable predictive power with a significantly simpler and more robust representation.

6.2.2. Validation against Reference Implementations

To verify the correctness of our custom Primal SVM implementation, we benchmarked its performance against the `LinearSVC` and `SGDClassifier` estimators from the Scikit-Learn library (Pedregosa et al., 2011). Our Squared Hinge (BFGS) solver achieved a test accuracy of 0.812 and identified 23,597 support vectors, closely matching the `LinearSVC` baseline (Accuracy: 0.814, Support Vectors: 23,640). The minor deviation ($\approx 0.2\%$) is attributable to differences in loss scaling (mean vs. sum) and solver termination criteria. Furthermore, our custom Stochastic Gradient Descent (SGD) implementation yielded a test accuracy of 0.804 with 9,769 support vectors, effectively replicating the behavior of Scikit-Learn's `SGDClassifier` (Accuracy: 0.807, Support Vectors: 10,036). The consistent reduction in support vectors ($\approx 58\%$ fewer than the smooth solvers) across both custom and reference implementations confirms that the observed sparsity is a fundamental property of the L1 Hinge loss, validating the correctness of our sub-gradient derivation and optimization logic.

7. Model Comparison

Finally, we compared the overall performance of all three models (GNB, Logistic Regression, and SVM) on the test set and synthesized our findings.

Table 4. Overall Performance Comparison on Test Set

Model	Accuracy	Optimization Cost
Gaussian NB	0.771	**None (Closed-form)**
Logistic Reg. (L2)	0.806	Low (BFGS: 10 iters)
SVM (Squared Hinge)	**0.812**	Low (BFGS: 18 iters)

7.1. Generative vs. Discriminative Approaches

The most interesting result is the performance gap between the statistical baseline and the optimization-based models. Both Logistic Regression (80.6%) and SVM (81.2%) significantly outperformed Gaussian Naive Bayes (77.1%). This result comes from two fundamental theoretical differences:

- **Violation of Independence Assumption:** GNB assumes that all features are conditionally independent given the class. However, our EDA revealed strong correlations among financial features (e.g., between `bill_amt` variables). GNB fails to capture these joint effects, accumulating estimation errors. In contrast, optimization-based models (LR, SVM) learn weights that account for these correlations directly, allowing them to construct a more accurate boundary.
- **Direct vs. Indirect Learning:** GNB is a generative model that estimates the full distribution $P(x \mid y)$ to compute the posterior. If the Gaussian assumption is inaccurate, the model's performance degrades. In contrast, optimization-based models are discriminative, mapping inputs directly to outputs $P(y \mid x)$ (or a decision score) by focusing on minimizing classification error. By addressing the problem directly, these models tend to achieve better generalization.

7.2. Linear Separability

Logistic Regression and SVM performed similarly (80.6% vs 81.2%). This suggests that the decision boundary for credit card default is largely linear. Since both are linear classifiers, they converged to similar solutions.

However, SVM achieved a slight but consistent edge (+0.6%). We believe this is due to the difference in objective functions. While Logistic Regression minimizes probabilistic loss (Log-Loss), SVM maximizes the geometric margin. This margin-based approach often provides slightly better robustness and generalization on unseen data, pushing the decision boundary away from ambiguous points.

7.3. Cost-Benefit Tradeoff

Lastly, we analyzed the trade-off between computational cost and accuracy. GNB offers the advantage of instant training through closed-form equations. However, our experiments showed that with efficient solvers like BFGS, the cost of optimization is negligible (converging in only 10–18 iterations). Given the accuracy gain (+3–4%) for a critical financial task like default prediction, the iterative optimization approach is more preferable to the simple statistical baseline.

8. Conclusion & Discussion

8.1. Conclusion: The Interplay of Formulation and Optimization

This project demonstrates that effective machine learning lies at the intersection of statistical estimation and numerical optimization. Our comparative analysis reveals that there is no universal gold standard estimator; rather, the optimal choice is strictly dependent on the underlying data structure and the specific constraints of the application domain.

While the generative baseline (Gaussian Naive Bayes) offered computational efficiency, its reliance on the assumption of feature independence proved detrimental in a financial context characterized by correlated variables. In contrast, the discriminative power of optimization-based models (Logistic Regression and SVM) successfully captured these joint effects, yielding significantly higher accuracy. Furthermore, the marginal superiority of the SVM ($+0.6\%$) highlights that formulation matters: by minimizing a geometric margin rather than a probabilistic loss, the SVM achieved greater robustness against decision boundary ambiguity.

Ultimately, we conclude that optimization is not merely a backend process but a fundamental component of model behavior. The choice between a *dense* solver (BFGS) for precision and speed, versus a *sparse* solver (SGD) for robustness and memory efficiency, must be informed by domain knowledge, balancing the cost of computation against the value of predictive accuracy.

8.2. Future Directions

To extend the scope of this framework and address the limitations observed, future work should focus on the following areas:

- **Dual Formulation and KKT Analysis:** While this study focused on the Primal SVM for computational efficiency on large N, implementing the Dual formulation via Quadratic Programming (QP) would offer deeper theoretical insights. Solving the Dual would allow for the direct calculation of Lagrange multipliers (α_i), enabling a precise mathematical verification of the Support Vectors identified geometrically in the Primal approach and verifying the Karush-Kuhn-Tucker (KKT) conditions.

- **Non-Linearity and Kernel Methods:** The performance ceiling observed around 81% for both Logistic Regression and SVM suggests that the dataset is not perfectly linearly separable. Future iterations should incorporate Kernel Methods (e.g., Radial Basis Function or Polynomial kernels) to project features into higher-dimensional spaces. This would allow the model to capture complex, non-linear dependencies between repayment history and demographic factors that linear boundaries fail to resolve.
- **Scalable Second-Order Optimization (L-BFGS):** Our BFGS implementation demonstrated superior convergence speed but struggled with stability on the dense Squared Hinge objective. For larger datasets where storing the full inverse Hessian approximation is prohibitive, implementing Limited-memory BFGS (L-BFGS) would be a critical step. This would maintain the superlinear convergence benefits of quasi-Newton methods while reducing memory complexity from $O(d^2)$ to $O(md)$, ensuring scalability for high-dimensional feature spaces.

9. Team Contributions

- **Jihoon Choi**: Developed the Primal Support Vector Machine (SVM) framework from the scratch, incorporating both smooth and non-smooth loss formulations. Implemented and benchmarked advanced optimization strategies, including BFGS and annealing-based SGD, to analyze the trade-offs between numerical efficiency and sparsity, and validated all custom solvers against `scikit-learn` baseline `SGDClassifier`.
- **Dae Young Kim**: Conducted exploratory data analysis and preprocessing. Implemented Gaussian Naive Bayes from scratch, evaluated the performance, and validated the implementation against `scikit-learn`. Experimented optimization algorithms for L2-regularized Logistic Regression, analyzed and compared their performance and efficiency, and validated all results against `scikit-learn`.

References

Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, NY, 2nd edition, 2009.

Kochenderfer, M. J. and Wheeler, T. A. *Algorithms for Optimization*. MIT Press, Cambridge, MA, 2019.

Nocedal, J. and Wright, S. J. *Numerical Optimization*. Springer, New York, NY, 2nd edition, 2006.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Yeh, I.-C. and hui Lien, C. Default of credit card clients dataset. https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients, 2009. Published with the paper: The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients in Expert Systems with Applications.